

Sardar Bilal Naeem

24P-3066

Hassan Israr

24P-3001

SE-3A

Loan Calculator Project Report

Project Overview

The **Loan Calculator** is a C++17-based financial tool designed to calculate various loan-related metrics with precision. It computes **monthly payments (EMI)**, **remaining loan balance**, **total number of payments**, **original loan amount**, and **effective interest rates**, including optional opening fees. The calculator validates user inputs to prevent incorrect computations and supports flexible inputs such as **initial payment**, **opening fees**, **loan tenure**, and **interest rate**.

The project supports two modes of operation:

- **Graphical User Interface (GUI)**: Built with Qt, allows interactive data entry and visualization of results.
- **Command-Line Interface (CLI)**: Supports direct computation with different command-line options, enabling scripting or automation.

This dual-mode design ensures accessibility for both end-users and developers, while **GoogleTest unit tests** provide stability and correctness, and **Doxygen documentation** ensures maintainability.

Project Components

The project consists of the following core components:

- **LoanCalculator Class**: Handles all financial calculations and input validation.
- **Qt GUI Application (LoanCalcQtMainWindow)**: Enables visual interaction for users.
- **Command-Line Interface**: Executes calculations using command-line arguments.
- **Configuration File (config.txt)**: Stores default values for loan amount, interest, and fees.
- **Unit Testing**: GoogleTest framework ensures correctness for normal EMI, invalid inputs, and large tenures.
- **Documentation**: Doxygen generates comprehensive HTML documentation for all functions and classes.

Code Snippets of Bug Fixes and Refactorings

Bug 1: Invalid Inputs

Problem: Users could pass negative loan amount, negative interest, zero/negative payment, or invalid periods, causing **division by zero or domain errors**.

Fix: Added validation inside setter functions:

```
inline void setAmount(long double A) {
    if(A <= 0) throw std::invalid_argument("Loan amount must be > 0");
    amount_ = A; amountSet_ = true;
}

inline void setInterest(long double i) {
    if(i <= 0) throw std::invalid_argument("Interest must be > 0");
    interest_ = i;
    interestPeriodic_ = i / 100.0 / 12.0;
    interestSet_ = true;
}

inline void setPeriodTotal(unsigned long N) {
    if(N == 0) throw std::invalid_argument("Total period must be > 0 months");
    periodTotal_ = N; periodTotalSet_ = true;
}
```

Bug 2: Large Tenure Overflow

Problem: Calculating $(1+i)^N$ for long tenures (e.g., 30 years = 360 months) with `float` could overflow and produce incorrect results.

Fix: Switched to **long double** for all core formulas:

```
long double i = (long double)interestPeriodic_;

long double totalAmount = (long double)amount_ - initialPayment_;

totalAmount += openingFee_ + (totalAmount * (openingPercent_/100.0));

long double denom = 1.0L - powl(1.0L + i, -(long double)periodTotal_);

long double result = (i * totalAmount) / denom;

return result;
```

Bug 3: Hardcoded Default Values

Problem: Opening fee, opening percent, and interest values were hardcoded in code.

Fix: Values now read from **config.txt**:

```
ifstream f("config.txt");

if(f.good()){

    string key; long double val;

    while(f >> key >> val){

        if(key == "default_interest") calculator.setInterest(val);

        else if(key == "default_opening_fee") calculator.setOpeningFee(val);

        else if(key == "default_opening_percent") calculator.setOpeningPercent(val);

    }

}
```

Bug 4: Invalid Number of Payments

Problem: Formula $N = -\log(1 - i \cdot A / P) / \log(1 + i)$ fails if $1 - i \cdot A / P \leq 0$, causing domain errors.

Fix: Added checks for valid inputs:

```
long double arg = 1.0L - (i * A / P);

if(!(arg > 0.0L))

    throw std::invalid_argument("Payment too small. Loan will never be repaid.");

long double numerator = -logl(arg);

long double denominator = logl(1.0L + i);

if(!(denominator > 0.0L))

    throw std::invalid_argument("Invalid periodic interest value.");
```

```
long double result = numerator / denominator;
```

```
return result;
```

Refactorings

- **LoanCalculator.cpp:** All calculation functions refactored to use `long double` and added input validation.
- **LoanCalculator.h:** Cleaned setters/getters and removed hardcoded values.
- **Main Application:** Command-line parser and GUI load config dynamically from `config.txt`.
- **Unit Tests:** GoogleTest updated to cover normal EMI, invalid inputs, and large tenure scenarios.

All functions now handle edge cases safely and prevent runtime errors.

Test Output

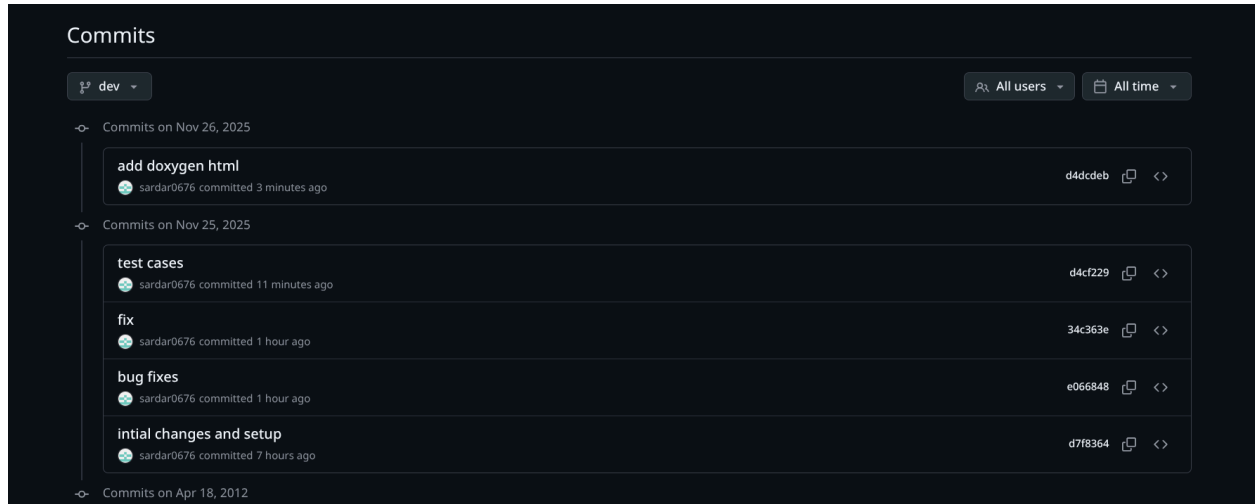
All unit tests executed with GoogleTest:

- **Normal EMI Calculation:** Passed
- **Invalid Input Handling:** Passed (exception thrown for invalid inputs)
- **Large Tenure:** Passed (no overflow, valid EMI computed)

```
• bali@fedora:~/Downloads/legacy-calc-2009-dev$ g++ -std=c++17 test.cpp LoanCalculator.cpp -lgtest -lgtest_main -lpthread -o testrun
• bali@fedora:~/Downloads/legacy-calc-2009-dev$ ./testrun
Running main() from /build/Build/gtest-1.15.2-build/googletest-1.15.2-build/googletest/src/gtest_main.cc
[=====] Running 3 tests from 1 test suite.
[-----] Global test environment set-up.
[-----] 3 tests from LoanCalculatorTest
[ RUN      ] LoanCalculatorTest.NormalEmiCalculation
[ OK       ] LoanCalculatorTest.NormalEmiCalculation (0 ms)
[ RUN      ] LoanCalculatorTest.InvalidInputThrowsException
[ OK       ] LoanCalculatorTest.InvalidInputThrowsException (0 ms)
[ RUN      ] LoanCalculatorTest.LargeTenureDoesNotOverflow
[ OK       ] LoanCalculatorTest.LargeTenureDoesNotOverflow (0 ms)
[-----] 3 tests from LoanCalculatorTest (0 ms total)

[-----] Global test environment tear-down
Ln 46, Col 2  Spaces: 4  UTF-8  LF  {} C++  Fini
```

Git Commit History



The screenshot shows the 'Commits' page for the 'dev' branch. At the top, there's a header with 'dev' and filters for 'All users' and 'All time'. Below this, the commit history is displayed, grouped by date. The most recent commit is 'add doxygen html' by sardar0676, committed 3 minutes ago, with hash d4dcdeb. The next group of commits is from Nov 25, 2025, including 'test cases' (d4cf229), 'fix' (34c363e), 'bug fixes' (e066848), and 'intial changes and setup' (d7f8364). The history continues with commits from Apr 18, 2012.

Commit Message	Author	Committed	Hash	Actions
add doxygen html	sardar0676	committed 3 minutes ago	d4dcdeb	View Compare
Commits on Nov 25, 2025				
test cases	sardar0676	committed 11 minutes ago	d4cf229	View Compare
fix	sardar0676	committed 1 hour ago	34c363e	View Compare
bug fixes	sardar0676	committed 1 hour ago	e066848	View Compare
intial changes and setup	sardar0676	committed 7 hours ago	d7f8364	View Compare
Commits on Apr 18, 2012				

Link:

<https://github.com/sardar0676/legacy-calc-2009/tree/dev>